

## Visualization

Prof. Bernhard Schmitzer, Uni Göttingen, summer term 2026

## Exam 2026-09-21

### General remarks.

- The exam is built on two datasets provided in `data_X/`. Each dataset is briefly described in a corresponding meta file `meta.md` and the reference to the original source is given. Acquaint yourselves with all datasets and their conventions.
- Work with Python and jupyter notebooks, e.g. via the GWDG jupyter server at <https://jupyter-cloud.gwdg.de/>.
- Your submission should consist of (multiple) notebook files (optionally in text format, e.g. via jupyter text), combined into a single zip/tar archive.
- Do not include the data files in your submission.
- Submit via StudIP, similar to the problem sheets.
- **Submission deadline: 2026-10-02 18:00**
- Indicate your enrolment numbers clearly at the beginning of each notebook file.
- Groups of two students do not have to complete problems 1.2 and 1.8. For groups of three, it is not advisable to simply assign these two sub-problems to the third person. Genuinely working as a team on problem 1 will be much more efficient.
- To judge the expected amount of work in the assignments, an approximate number of suggested charts is given in parentheses. There is no single right solutions to the assignments. Sometimes more or even less charts may be appropriate, depending on your approach. **A single chart may consist of small multiples.** Including excessively many charts in your submission without a nuanced discussion will be considered as poor judgment.
- When writing text, be concise. Do not include extensive, verbose discussions of the data or plots beyond the questions in the notebooks.
- If you do not understand a problem description, do not hesitate to contact `schmitzer@cs.uni-goettingen.de`.

### Problem 1: Munich public transport network.

This exercise deals with the public transport network of Munich as operated by [MVG](#). Data on stops, routes, and the schedule is contained in `./data_mvg/`.

1. Familiarize yourself with the [GTFS](#) format, in particular for the included files `routes.txt`, `stops.txt`, `trips.txt`, `stop_times.txt` and import these files as pandas dataframes.
2. Visualize the relational model underlying a GTFS-format database for the following tables/files: `routes`, `stops`, `trips`, `stop_times`, `calendar`, `transfers`. Other tables, not listed here, may be ignored. (1 chart)

3. Determine for each stop, by which routes it is served. Which stops are served by the most routes? What is the distribution over the number of routes that serve the stops? What are the contributions by different route types (as in: subway, bus, tram,...)? (2 charts)
4. Obtain a list of all individual calls made by any trip at any stop, on a weekday, and on a weekend day. These calls are represented by rows in the `stop_times` table (in proper combination with the other tables). Show the temporal distributions of calls throughout the day. Also resolve this by route type. (1-2 charts)
5. Revisit the questions of problem 1.3, but now consider individual stops of vehicles (only for weekdays), not merely being served by a route. (2 charts)
6. Using the available data (especially in `stop_times.txt`) determine the joint distribution of times and distances for vehicles between individual stops. Show this joint distribution, of course resolved by route type. (1-2 charts)
7. With the data from the previous problem, compute the corresponding velocities of vehicles and show their distribution by route type. (1 chart)
8. Create an animation of all trips on a weekday in Munich. The precise trajectories of the vehicles is not important, simply assume that vehicles move between stops on straight lines with constant speed, according to the schedule given in `stop_times.txt` and locations in `stops.txt`. It may be advisable that this animation is rendered and stored as a rasterized video file, as it will likely be too large to handle interactively. (1 chart)

## Problem 2: German census data 2022.

This problem analyzes some aspects of the German national census data of 2022 contained in `./data_census/`.

1. Familiarize yourself with the data format and import the census and location files as pandas dataframes.
2. Show distribution of municipality sizes (=number of residents) in Germany. Also create a chart which can answer the question which fraction of people lives in municipalities up to a given size. (2 charts)
3. For each municipality compute the overall fraction of women among all residents. Show how this fraction is distributed over municipality size. Make this plot dynamically explorable, such that curious datapoints can be identified and examined. (1 chart)
4. Refine the previous chart by performing the same analysis on individual age brackets. (1 chart)
5. Show the prevalence of different marital statuses over age brackets and gender. (1 chart)
6. For each municipality compute the relative distribution of residents over the age brackets. Then compute the cumulative distribution over age brackets from youngest to oldest (normalized such that the last entry is one, not the total number of residents). Now represent each municipality by its 11-dimensional cumulative distribution vector. Perform a principal component analysis on this point cloud and discuss the effective dimension of this dataset. Provide a chart to support your conclusion. (1-2 charts)

7. Visualize the average age distribution and the first three PCA eigenvectors and briefly explain which trends they represent. Consider inverting the cumulative sum operation for this visualization. (1 chart)
8. Project all municipalities into the orthonormal basis spanned by the first three principal components. Analyze qualitatively, by means of suitable visualizations, if there are dependencies between the PCA coordinates, the municipality size and location (longitude, latitude, or both). (3-4 charts)